# TP n°1 – Test de primalité

Étude des algorithmes et mesures expérimentales

HAMMAL Zakaria TRIBECHE Rayan NAIT MIHOUB Imad
FERGUENE Abderraouf

4 novembre 2026

## Table des matières

**Introduction** **2**

**1 Algorithme A1 : test naïf jusqu'à $N-1$** **2**
1.1 Algorithme correspondant . . . 2
1.2 Complexité théorique au pire cas . . . 2
1.3 Vérification des entrées et mesure du temps d'exécution . . . 3

**2 Algorithme A2 : tester jusqu'à $N/2$** **5**
2.1 Algorithme correspondant . . . 5
2.2 Complexité théorique au pire cas . . . 5

**3 Algorithme A3 : tester jusqu'à $\sqrt{N}$** **6**
3.1 Algorithme correspondant . . . 6
3.2 Complexité théorique au pire cas . . . 7

**4 Algorithme A4 : tester jusqu'à $\sqrt{N}$ avec les nombres impairs seulement** **7**
4.1 Algorithme correspondant . . . 7
4.2 Complexité théorique au pire cas . . . 8

**5 Comparaison des algorithmes pour les différents jeux de tests** **9**

**Conclusion** **14**

# Introduction

L'objectif de ce TP est de tester différentes méthodes pour le même problème (tester si un nombre entier naturel est premier) et d'apprendre à mesurer le temps d'exécution d'un programme. On implémente quatre algorithmes en langage C, puis on les compare à l'aide des fonctions de gestion du temps de la bibliothèque `time.h`, en utilisant les cinq jeux de données préparés dans le TP n°0 : `Random100`, `Random1000`, `Test-1`, `Test-2` et `Test-3`.

Rappel : un nombre entier naturel $N$ est premier s'il n'a que deux diviseurs, 1 et $N$ lui-même.

Les quatre algorithmes étudiés suivent une progression naturelle :

— **A1** : test naïf jusqu'à $N-1$ ;
— **A2** : test jusqu'à $N/2$ (tout diviseur propre est $\leq N/2$) ;
— **A3** : test jusqu'à $\sqrt{N}$ (les diviseurs vont par paires) ;
— **A4** : test des seuls diviseurs impairs jusqu'à $\sqrt{N}$ (tout premier $> 2$ est impair).

# 1 Algorithme A1 : test naïf jusqu'à $N-1$

Dans les pseudo-codes ci-dessous, le cas $N < 2$ est rejeté. Les bornes correspondent aux tests effectués dans le programme.

## 1.1 Algorithme correspondant

A1 teste tous les diviseurs possibles compris entre 2 et $N-1$. Si aucun ne divise $N$, le nombre est premier.

**Algorithme 1** Algorithme A1 : test naïf jusqu'à $N-1$

**Entrées :** $N$ : entier
**Sorties :** `est_premier` : booléen
**Variables :** `trouv` : booléen ; $i$ : entier

```
1 : Si N < 2 Alors
2 :     trouv ← vrai
3 : Sinon
4 :     i ← 2
5 :     trouv ← faux
6 :     Tant que i < N et trouv = faux Faire
7 :         Si N mod i = 0 Alors
8 :             trouv ← vrai
9 :         Sinon
10 :            i ← i + 1
11 :        Fin Si
12 :    Fin Tant que
13 : Fin Si
14 : Retourner ¬ trouv
```

## 1.2 Complexité théorique au pire cas

Il faut bien noter que, dans cet algorithme tel que présenté dans ce rapport, on a ajouté une action de vérification ($N < 2$), ce qui n'est pas le cas dans le code `.c` attaché, car on utilise un jeu de données que l'on a généré dans le TP n°0.

**Calcul détaillé de la complexité (pire cas).**

— Vérification $N < 2$ : $1u$ (on ne prend pas l'instruction `trouv` ← **vrai** car il s'agit de la complexité au pire cas).
— **i ← 2** : $1u$.

— **trouv** $\leftarrow$ **faux** : $1u$.
— Condition de boucle **(i < n) et (trouv == faux)** : $1u + 1u + 1u = 3u$.
— **N mod i = 0** : $1u + 1u = 2u$.

Dans le pire cas, la condition de sortie n'est jamais vérifiée à l'intérieur de la boucle, donc on comptabilise à chaque itération le bloc *sinon* :

— **i** $\leftarrow$ **i + 1** : $1u + 1u = 2u$ ;
— Nombre d'itérations :

$$\text{nbritr} = \frac{\text{bornsup} - \text{borninf}}{\text{pas}} = \frac{n-2}{1} = n - 2.$$

Le nombre d'instructions dans la boucle est égal au nombre d'instructions d'une itération multiplié par le nombre d'itérations, plus le nombre d'instructions de la dernière vérification : $7(n-2) + 3 = 7n - 11$.

Enfin, **retourner** $\neg$ **trouv** : $1u + 1u = 2u$.

Alors :

$$A_1(n) = 1 + 1 + 7n - 11 + 2 = 7n - 7.$$

La complexité au pire cas est donc : $O(N)$.

### 1.3 Vérification des entrées et mesure du temps d'exécution

Cette sous-section répond aux questions **a** à **e** de l'énoncé.

**Question a : vérification de la primalité.** Tous les nombres proposés dans l'énoncé du TP sont bien premiers, comme le montre la colonne « Premier » du tableau ci-dessous (toutes les lignes indiquent « Oui »).

**Question b : mesure des temps d'exécution.** Le tableau présente le temps moyen d'une exécution en secondes. Les mesures ont été produites par `tp1_runner` ; les tableaux CSV détaillés se trouvent dans le dossier `results/`.

| $N$ | A1 (s) | Premier |
|---:|---:|:---:|
| 1 000 003 | 0,002 046 906 25 | Oui |
| 2 000 003 | 0,004 110 625 | Oui |
| 4 000 037 | 0,008 209 75 | Oui |
| 8 000 009 | 0,016 371 75 | Oui |
| 16 000 057 | 0,033 039 | Oui |
| 32 000 011 | 0,066 288 | Oui |
| 64 000 031 | 0,135 053 | Oui |
| 128 000 003 | 0,259 705 | Oui |
| 256 000 001 | 0,524 776 | Oui |
| 512 000 009 | 1,050 29 | Oui |
| 1 024 000 009 | 2,103 033 | Oui |
| 2 048 000 011 | 4,210 006 | Oui |

TABLE 1 – Temps d'exécution moyens mesurés pour l'algorithme A1.

**Question c : observation des mesures.** On remarque d'abord que, dans l'échantillon proposé, chaque nombre $N$ est approximativement le double du précédent. On remarque ensuite que le temps d'exécution s'incrémente d'une manière approximativement linéaire : quand $N$ double, le temps double aussi. Ce comportement se vérifie en calculant le rapport $\Delta t / \Delta N$ entre deux lignes consécutives, qui reste approximativement constant autour de 2,05 ns par unité :

| $N$ | t = A1 (s) | $\Delta N$ | $\Delta t/\Delta N$ (ns) |
|---:|---:|---:|---:|
| 1 000 003 | 0,002 046 906 25 | — | — |
| 2 000 003 | 0,004 110 625 | 1 000 000 | 2,064 |
| 4 000 037 | 0,008 209 75 | 2 000 034 | 2,050 |
| 8 000 009 | 0,016 371 75 | 3 999 972 | 2,041 |
| 16 000 057 | 0,033 039 | 8 000 048 | 2,083 |
| 32 000 011 | 0,066 288 | 15 999 954 | 2,078 |
| 64 000 031 | 0,135 053 | 32 000 020 | 2,149 |
| 128 000 003 | 0,259 705 | 63 999 972 | 1,948 |
| 256 000 001 | 0,524 776 | 127 999 998 | 2,071 |
| 512 000 009 | 1,050 29 | 256 000 008 | 2,053 |
| 1 024 000 009 | 2,103 033 | 512 000 000 | 2,056 |
| 2 048 000 011 | 4,210 006 | 1 024 000 002 | 2,058 |

TABLE 2 – Évolution du rapport $\Delta t/\Delta N$ pour l'algorithme A1.

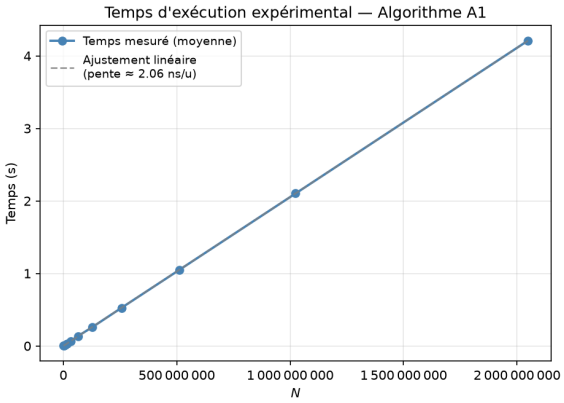
**Question d : comparaison théorie / mesures.** Pour comparer la complexité théorique et les mesures expérimentales, on calcule le nombre théorique d'instructions $T(N) = 7N - 7$ et on forme le ratio $t/T(N)$, exprimé en nanosecondes par unité d'instruction.

| $N$ | t = A1 (s) | $T(N) = 7N - 7$ (u) | $t/T(N)$ (ns/u) |
|---:|---:|---:|---:|
| 1 000 003 | 0,002 046 906 25 | 7 000 014 | 0,2924 |
| 2 000 003 | 0,004 110 625 | 14 000 014 | 0,2936 |
| 4 000 037 | 0,008 209 75 | 28 000 252 | 0,2932 |
| 8 000 009 | 0,016 371 75 | 56 000 056 | 0,2924 |
| 16 000 057 | 0,033 039 | 112 000 392 | 0,2950 |
| 32 000 011 | 0,066 288 | 224 000 070 | 0,2959 |
| 64 000 031 | 0,135 053 | 448 000 210 | 0,3015 |
| 128 000 003 | 0,259 705 | 896 000 014 | 0,2898 |
| 256 000 001 | 0,524 776 | 1 792 000 000 | 0,2928 |
| 512 000 009 | 1,050 29 | 3 584 000 056 | 0,2930 |
| 1 024 000 009 | 2,103 033 | 7 168 000 056 | 0,2934 |
| 2 048 000 011 | 4,210 006 | 14 336 000 070 | 0,2937 |

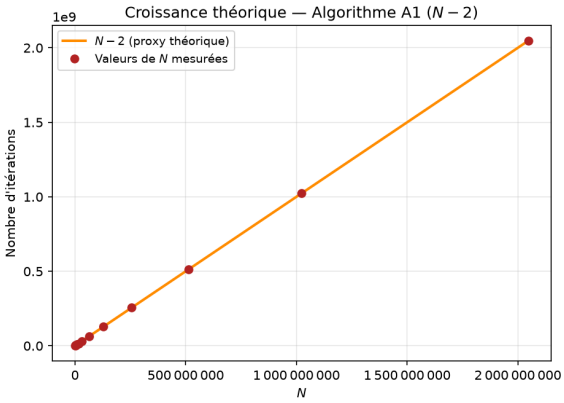
TABLE 3 – Comparaison entre la complexité théorique et les mesures expérimentales.

On remarque que le ratio $t/T(N)$ est approximativement constant, autour de 0,29 ns par unité d'instruction, donc on peut considérer que les prédictions théoriques sont compatibles avec les mesures expérimentales, et que la complexité observée est bien en $O(N)$.

**Question e : représentation graphique.** La figure ci-dessous présente, à gauche, le temps expérimental $T(N)$ en fonction de $N$, et à droite, la complexité théorique $N - 2$ en fonction de $N$.




Temps expérimental




Complexité théorique $N - 2$

FIGURE 1 – Résultats expérimentaux et croissance théorique de A1.

# 2 Algorithme A2 : tester jusqu'à $N/2$

Dans les pseudo-codes ci-dessous, le cas $N < 2$ est rejeté. Les bornes correspondent aux tests effectués dans le programme.

## 2.1 Algorithme correspondant

A2 teste tous les diviseurs possibles compris entre 2 et $N/2$. Si aucun ne divise $N$, le nombre est premier. En effet, tout diviseur $i$ de $N$ distinct de $N$ lui-même vérifie $i \leq N/2$, donc il est inutile de tester au-delà de cette borne.

**Algorithme 2** Algorithme A2 : test jusqu'à $N/2$

**Entrées :** $N$ : entier
**Sorties :** `est_premier` : booléen
**Variables :** `trouv` : booléen ; $i$ : entier

```
1 : Si N < 2 Alors
2 :     trouv ← vrai
3 : Sinon
4 :     i ← 2
5 :     trouv ← faux
6 :     Tant que i ≤ N/2 et trouv = faux Faire
7 :         Si N mod i = 0 Alors
8 :             trouv ← vrai
9 :         Sinon
10 :            i ← i + 1
11 :        Fin Si
12 :    Fin Tant que
13 : Fin Si
14 : Retourner ¬ trouv
```

## 2.2 Complexité théorique au pire cas

Il faut bien noter que, dans cet algorithme tel que présenté dans ce rapport, on a ajouté une action de vérification ($N < 2$), ce qui n'est pas le cas dans le code `.c` attaché, car on utilise un jeu de données que l'on a généré dans le TP n°0.

**Calcul détaillé de la complexité (pire cas).**

— Vérification $N < 2$ : $1u$.

— **i ← 2** : $1u$.
— **trouv ← faux** : $1u$.
— Condition **(i <= n/2) et (trouv == faux)** : $1u + 1u + 1u + 1u = 4u$.
— **N mod i = 0** : $1u + 1u = 2u$.

Dans le pire cas, on a la condition non vérifiée pour chaque itération, donc on comptabilise le bloc *sinon* :
— **i ← i + 1** : $1u + 1u = 2u$ ;
— Nombre d'itérations :

$$\text{nbritr} = \frac{\text{bornsup} - \text{borninf}}{\text{pas}} + \text{pas} = \frac{n/2 - 2}{1} + 1 = n/2 - 1.$$

Le nombre d'instructions dans la boucle est donc : $8(n/2 - 2) + 4 = 4n - 12$.
Enfin, **retourner ¬ trouv** : $1u + 1u = 2u$.
Alors :

$$A_2(n) = 1 + 1 + 4n - 12 + 2 = 4n - 8.$$

La complexité au pire cas est : $O(N)$, mais avec une constante deux fois plus petite que celle de A1.

# 3 Algorithme A3 : tester jusqu'à $\sqrt{N}$

Dans les pseudo-codes ci-dessous, le cas $N < 2$ est rejeté. Les bornes correspondent aux tests effectués dans le programme.

## 3.1 Algorithme correspondant

A3 teste tous les diviseurs possibles compris entre 2 et $\sqrt{N}$. Si aucun ne divise $N$, le nombre est premier.

**Algorithme 3** Algorithme A3 : test jusqu'à $\sqrt{N}$

**Entrées :** $N$ : entier
**Sorties :** `est_premier` : booléen
**Variables :** `trouv` : booléen ; $i$ : entier

```
1 : Si N < 2 Alors
2 :     trouv ← vrai
3 : Sinon
4 :     i ← 2
5 :     trouv ← faux
6 :     Tant que i ≤ √N et trouv = faux Faire
7 :         Si N mod i = 0 Alors
8 :             trouv ← vrai
9 :         Sinon
10 :            i ← i + 1
11 :        Fin Si
12 :    Fin Tant que
13 : Fin Si
14 : Retourner ¬ trouv
```

Grâce à la propriété des paires de diviseurs, A3 n'a besoin de tester que jusqu'à $\sqrt{N}$ : si $N$ admet un diviseur $d$, alors $N/d$ est aussi un diviseur de $N$, et l'un des deux est inférieur ou égal à $\sqrt{N}$. Son nombre d'essais est donc en $O(\sqrt{N})$, au lieu de $O(N)$. Le rapport entre ces deux quantités augmente avec $N$ : l'avantage d'A3 devient de plus en plus important lorsque les entrées grandissent.

### 3.2 Complexité théorique au pire cas

Il faut bien noter que, dans cet algorithme tel que présenté dans ce rapport, on a ajouté une action de vérification ($N < 2$), ce qui n'est pas le cas dans le code `.c` attaché, car on utilise un jeu de données que l'on a généré dans le TP n°0.

**Calcul détaillé de la complexité (pire cas).**

— Vérification $N < 2$ : $1u$.
— **i ← 2** : $1u$.
— **trouv ← faux** : $1u$.
— Condition **(i <= sqrt(n)) et (trouv == faux)** : $1u + 1u + 1u + 1u = 4u$.
— **N mod i = 0** : $1u + 1u = 2u$.

Dans le pire cas, on a la condition non vérifiée pour chaque itération, donc on comptabilise le bloc *sinon* :

— **i ← i + 1** : $1u + 1u = 2u$ ;
— Nombre d'itérations :

$$\text{nbritr} = \frac{\sqrt{n} - 2}{1} + 1 = \sqrt{n} - 1.$$

Le nombre d'instructions dans la boucle est donc : $8(\sqrt{n} - 2) + 4 = 8\sqrt{n} - 12$.
Enfin, **retourner ¬ trouv** : $1u + 1u = 2u$.
Alors :

$$A_3(n) = 1 + 1 + 8\sqrt{n} - 12 + 2 = 8\sqrt{n} - 8.$$

La complexité au pire cas est : $O(\sqrt{N})$.

## 4 Algorithme A4 : tester jusqu'à $\sqrt{N}$ avec les nombres impairs seulement

Dans les pseudo-codes ci-dessous, le cas $N < 2$ est rejeté. Les bornes correspondent aux tests effectués dans le programme.

### 4.1 Algorithme correspondant

A4 teste uniquement les diviseurs impairs compris entre 3 et $\sqrt{N}$, après avoir vérifié que $N$ n'est pas divisible par 2. Si aucun ne divise $N$, le nombre est premier.

**Algorithme 4** Algorithme A4 : test des diviseurs impairs jusqu'à $\sqrt{N}$

**Entrées :** $N$ : entier
**Sorties :** `est_premier` : booléen
**Variables :** `trouv` : booléen ; $i$ : entier

```
1 : Si N < 2 Alors
2 :     trouv ← vrai
3 : Sinon
4 :     Si N = 2 Alors
5 :         trouv ← faux
6 :     Sinon
7 :         Si N mod 2 = 0 Alors
8 :             trouv ← vrai
9 :         Sinon
10 :            i ← 3
11 :            trouv ← faux
12 :            Tant que i ≤ √N et trouv = faux Faire
13 :                Si N mod i = 0 Alors
14 :                    trouv ← vrai
15 :                Sinon
16 :                    i ← i + 2
17 :                Fin Si
18 :            Fin Tant que
19 :        Fin Si
20 :    Fin Si
21 : Fin Si
22 : Retourner ¬ trouv
```

### 4.2 Complexité théorique au pire cas

Il faut bien noter que, dans cet algorithme tel que présenté dans ce rapport, on a ajouté une action de vérification ($N < 2$), ce qui n'est pas le cas dans le code `.c` attaché, car on utilise un jeu de données que l'on a généré dans le TP n°0.

**Calcul détaillé de la complexité (pire cas).**

— Vérification $N < 2$ : $1u$.
— Vérification $N = 2$ : $1u$.
— **i ← 3** : $1u$.
— **trouv ← faux** : $1u$.
— Condition **(i <= sqrt(n)) et (trouv == faux)** : $1u + 1u + 1u + 1u = 4u$.
— **N mod i = 0** : $1u + 1u = 2u$.

Dans le pire cas, on a la condition non vérifiée pour chaque itération, donc on comptabilise le bloc *sinon* :

— **i ← i + 2** : $1u + 1u = 2u$ ;
— Nombre d'itérations :

$$\text{nbritr} = \frac{\sqrt{n} - 2}{1} + 2 = \sqrt{n}/2 - 2 + 1 = \sqrt{n}/2 - 1.$$

Le nombre d'instructions dans la boucle est donc : $8(\sqrt{n}/2 - 1) + 4 = 4\sqrt{n} - 4$.
Enfin, **retourner ¬ trouv** : $1u + 1u = 2u$.
Alors :

$$A_4(n) = 1 + 1 + 1 + 4\sqrt{n} - 4 + 2 = 4\sqrt{n} + 1.$$

La complexité au pire cas est : $O(\sqrt{N})$.

Grâce à la propriété selon laquelle les nombres premiers (à part 2) sont impairs, on ne vérifie que si $N$ est divisible par des nombres impairs, et dans le cas où $N$ est impair, ce qui donne une meilleure performance que A3.

## 5 Comparaison des algorithmes pour les différents jeux de tests

Les cinq figures qui suivent présentent, pour chaque jeu de données, le temps d'exécution mesuré pour les quatre algorithmes en fonction de $N$. **Les deux axes sont en échelle logarithmique.** Ce choix est nécessaire parce que, dans chacun des cinq jeux, les valeurs de $N$ couvrent plusieurs ordres de grandeur (de $10^2$ à $10^{11}$), et les temps d'exécution varient eux aussi sur plusieurs décades (de la microseconde à plusieurs minutes). Sur une échelle linéaire, les petites valeurs seraient écrasées contre l'axe des abscisses et deviendraient illisibles, tandis que seules les plus grandes valeurs seraient visibles ; l'échelle logarithmique compresse l'ensemble et permet au contraire de lire simultanément le comportement sur les petits nombres (qui apparaît alors dans la zone basse du graphique) et sur les grands nombres (qui apparaît dans la zone haute). Une droite de pente 1 en log-log correspond à une complexité en $O(N)$, et une droite de pente 1/2 correspond à une complexité en $O(\sqrt{N})$ : c'est ce qui permet de vérifier visuellement l'ordre des quatre algorithmes.

### Random100

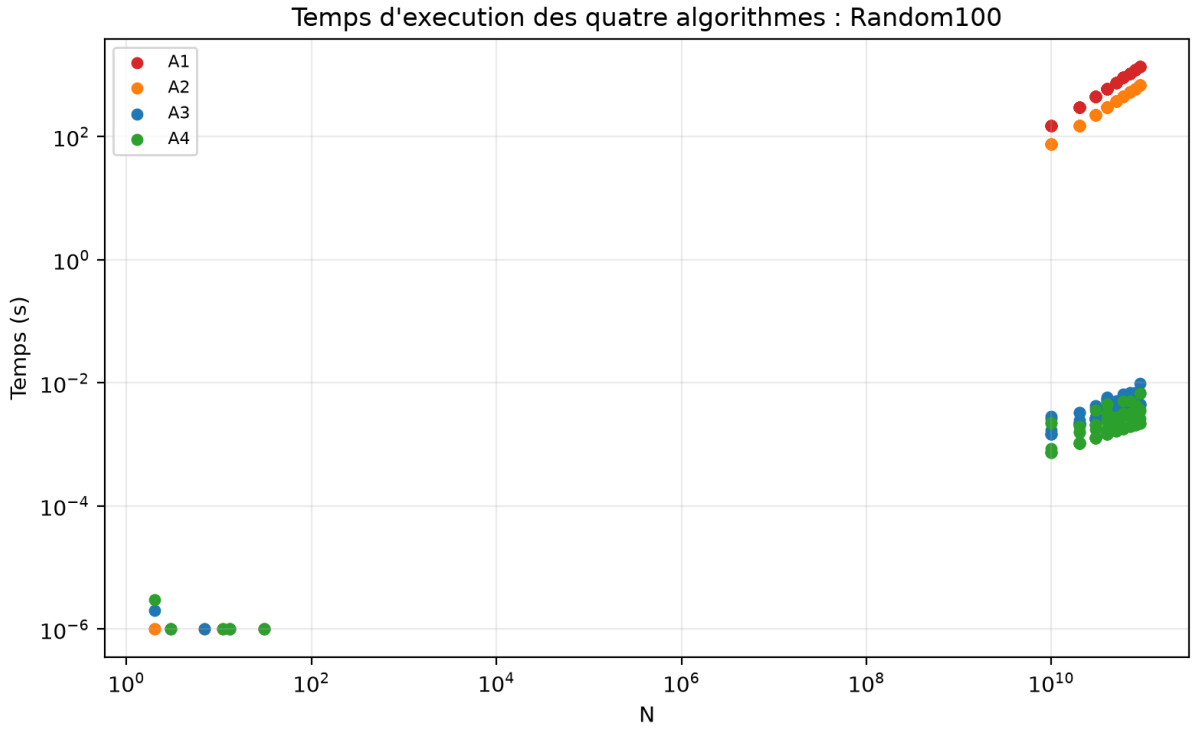



FIGURE 2 – Résultats expérimentaux pour le dataset `Random100` (échelle logarithmique sur les deux axes).

Sur `Random100`, qui contient des nombres allant de 2 à $9 \cdot 10^{10}$, les quatre algorithmes se distinguent très clairement : A1 est le plus lent, puis A2, puis A3, et A4 est le plus rapide. Comme les valeurs de $N$ s'étendent sur environ dix ordres de grandeur, seule l'échelle logarithmique permet de voir en même temps les temps très courts des petits nombres et les temps très longs des grands nombres ; sur une échelle linéaire, les nombres inférieurs à $10^8$ seraient invisibles. Pour un nombre

comme $9 \cdot 10^{10}$, A1 met environ 22 minutes, A2 environ 11 minutes, A3 quelques millisecondes et A4 un peu moins. L'écart entre les algorithmes en $O(N)$ (A1 et A2) et ceux en $O(\sqrt{N})$ (A3 et A4) devient très net dès que $N$ dépasse $10^{10}$, et se lit directement sur la figure comme la séparation verticale entre les deux familles de points.

### Random1000

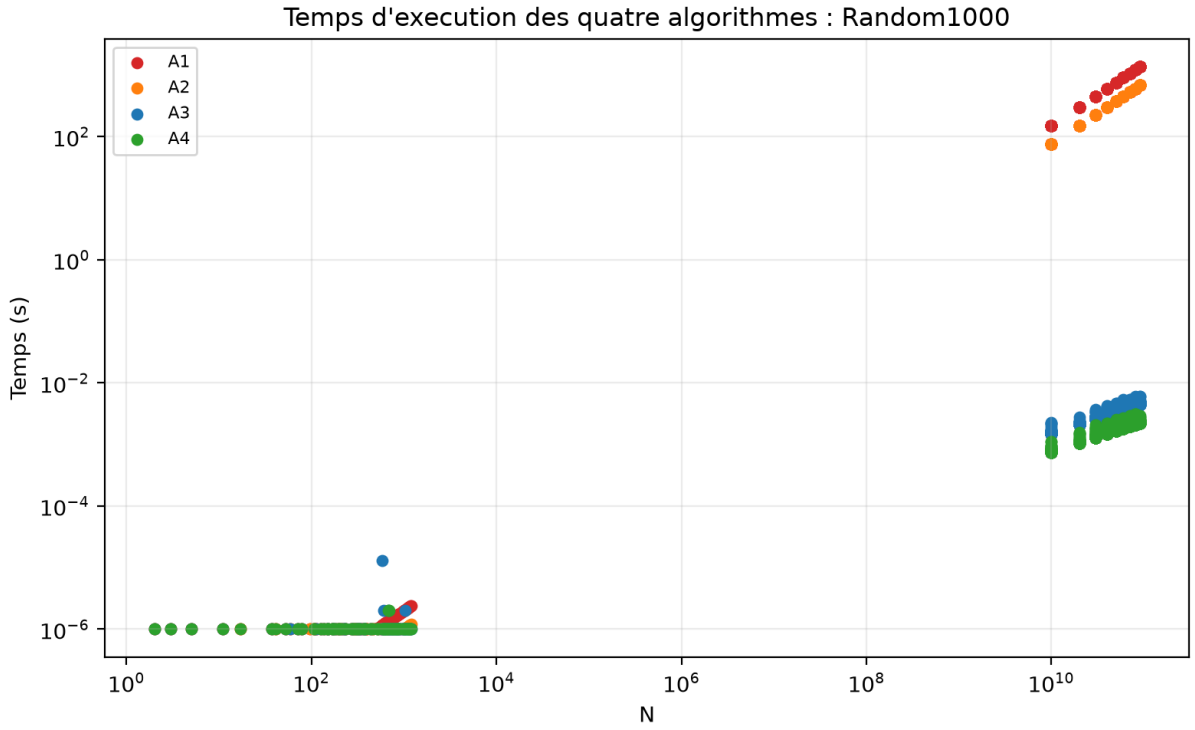



FIGURE 3 – Résultats expérimentaux pour le dataset `Random1000` (échelle logarithmique sur les deux axes).

Sur `Random1000`, qui contient 1000 nombres premiers de tailles très variées, on retrouve la même hiérarchie : $T_{A1}(N) > T_{A2}(N) > T_{A3}(N) > T_{A4}(N)$. Le nuage de points est plus dense, mais la séparation entre les algorithmes en $O(N)$ et ceux en $O(\sqrt{N})$ reste très nette. Là encore, l'échelle logarithmique est indispensable : avec 1000 points dont les valeurs de $N$ couvrent près de dix décades, une échelle linéaire condenserait la quasi-totalité des points près de l'origine et rendrait la hiérarchie invisible.

### Test-1

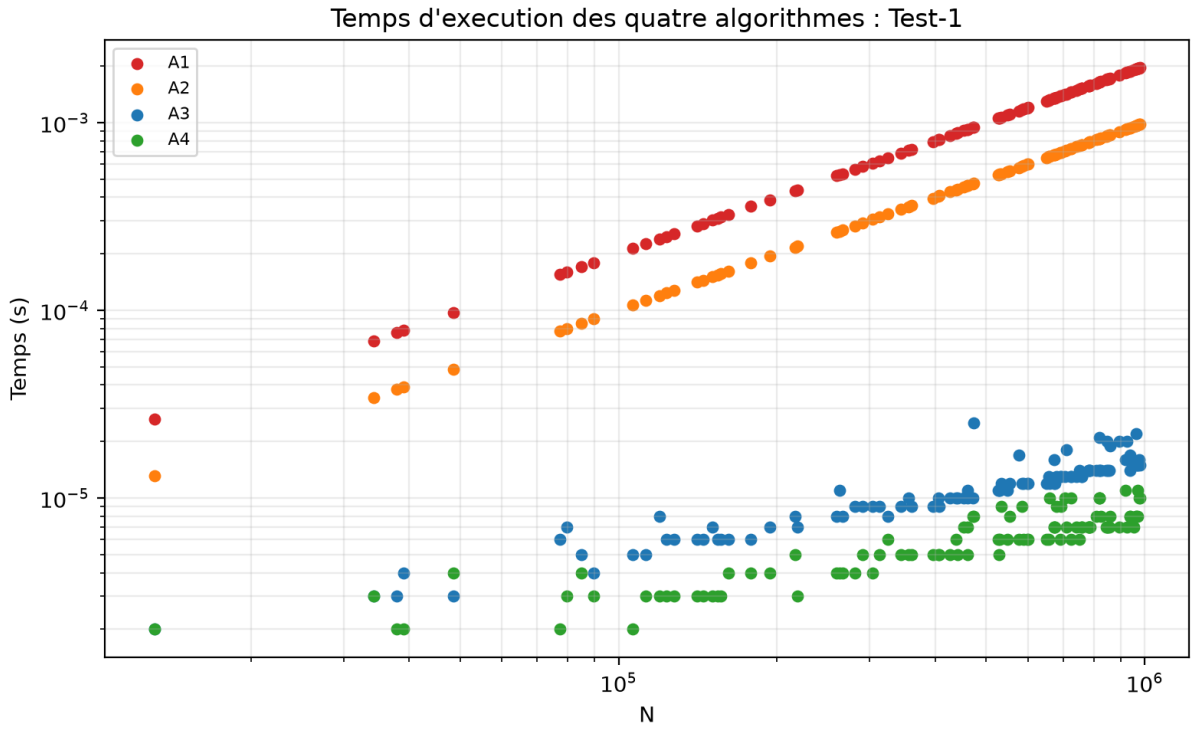



FIGURE 4 – Résultats expérimentaux pour le dataset `Test-1` (échelle logarithmique sur les deux axes).

Sur `Test-1`, dont les valeurs sont comprises entre $10^3$ et $10^6$, les quatre algorithmes restent sous la milliseconde, et la différence entre eux est peu visible à l'œil nu. L'ordre $T_{A1}(N) > T_{A2}(N) > T_{A3}(N) > T_{A4}(N)$ est toutefois respecté, conformément aux prédictions théoriques. Même si la plage est plus étroite que pour `Random100`, l'échelle logarithmique reste utile parce que les temps d'exécution varient, eux, sur plusieurs décades (de la microseconde à la milliseconde) et parce que l'écart entre A1 et A4 reste d'un facteur 10 environ.

### Test-2

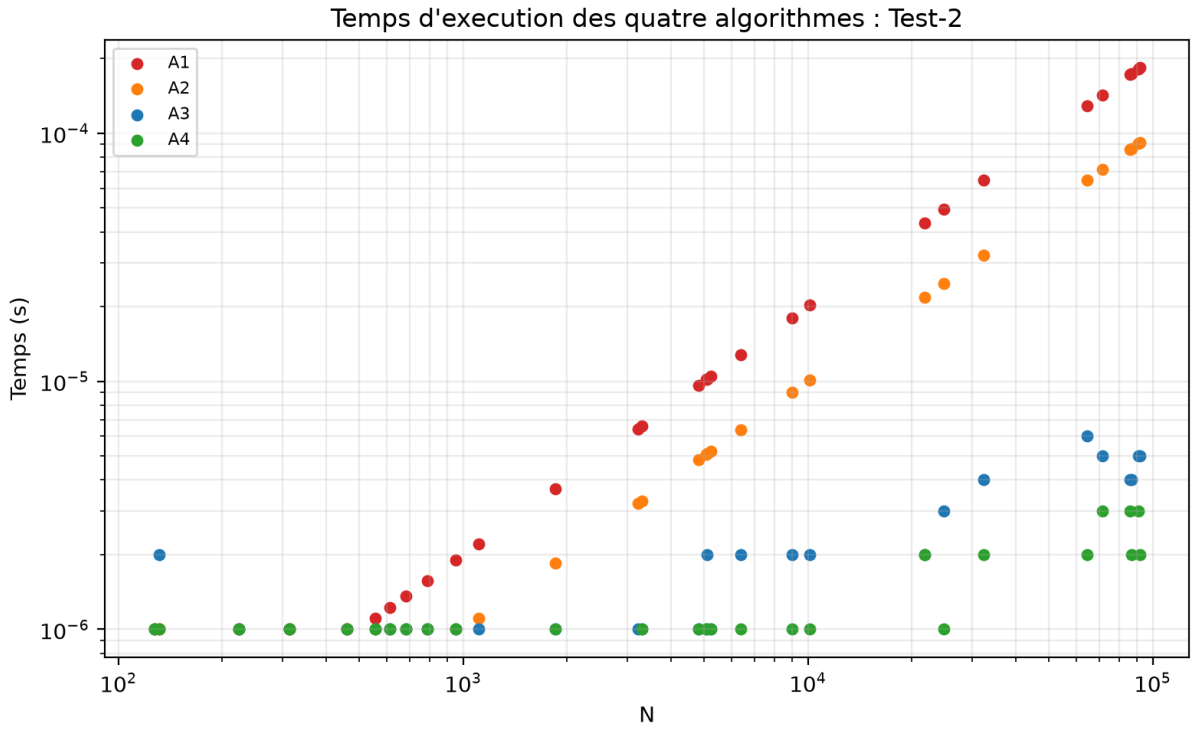



FIGURE 5 – Résultats expérimentaux pour le dataset `Test-2` (échelle logarithmique sur les deux axes).

Sur `Test-2`, qui contient des nombres à 3, 4 et 5 chiffres, les temps d'exécution sont également très petits. La différence entre les algorithmes est négligeable, même si A4 reste systématiquement le plus rapide sur les nombres à 5 chiffres. L'échelle logarithmique permet de distinguer les trois groupes (3, 4 et 5 chiffres) sur un même graphique : sur une échelle linéaire, les nombres à 3 et 4 chiffres seraient confondus avec l'origine et seuls les nombres à 5 chiffres seraient visibles.

### Test-3

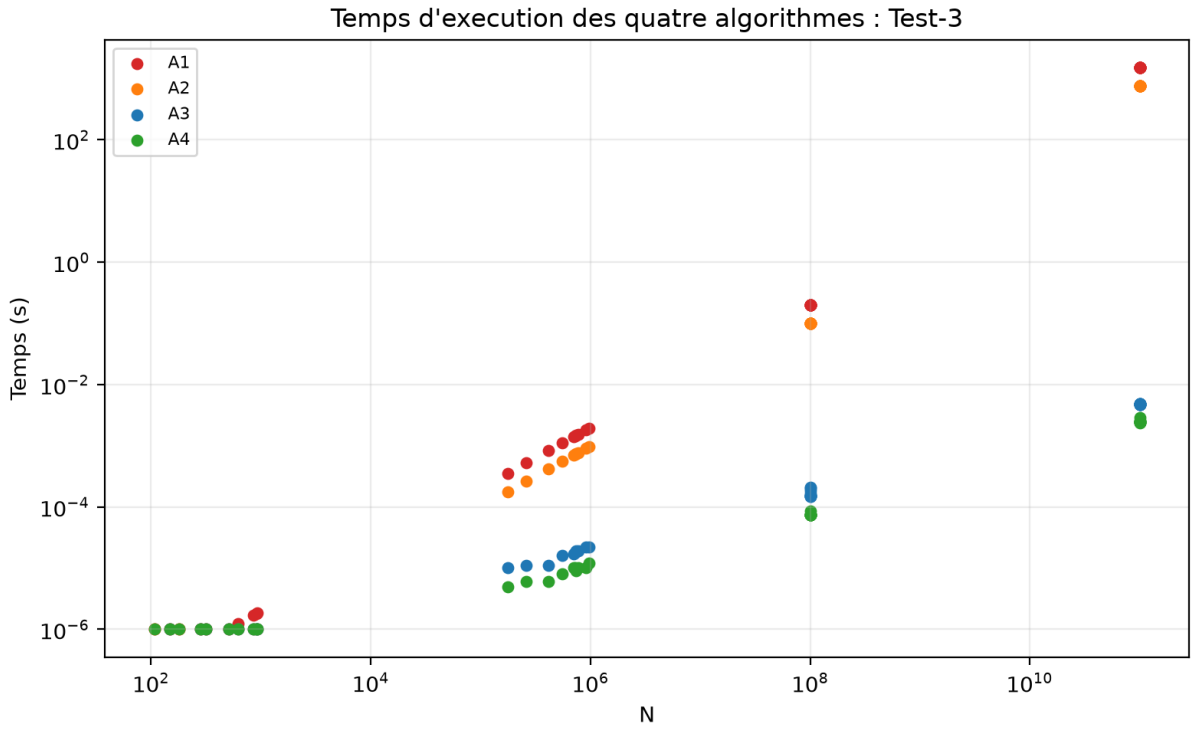



FIGURE 6 – Résultats expérimentaux pour le dataset `Test-3` (échelle logarithmique sur les deux axes).

Sur `Test-3`, qui contient des nombres à 3, 6, 9 et 12 chiffres, l'écart entre les algorithmes en $O(N)$ et ceux en $O(\sqrt{N})$ devient spectaculaire : sur les nombres à 12 chiffres (par exemple 100 000 000 003), A3 et A4 restent en quelques millisecondes, tandis qu'A1 et A2 dépasseraient plusieurs heures pour un seul nombre. C'est sur ce jeu que l'échelle logarithmique est la plus indispensable : les valeurs de $N$ s'étendent de $10^2$ à $10^{11}$ (neuf décades) et les temps correspondants de la microseconde à plusieurs heures (dix décades). Une échelle linéaire ne permettrait de voir que les nombres à 12 chiffres ; grâce à l'échelle logarithmique, on lit simultanément que, sur les petits nombres, les quatre algorithmes ont des temps très proches, et que, sur les grands nombres, A4 devient environ deux fois plus rapide que A3 et des millions de fois plus rapide que A1. Ces cas extrêmes ont donc été traités par extrapolation à partir du coût par itération mesuré sur A3 et A4.

### Commentaire général

Pour les petits nombres dans les cinq datasets, la différence de complexité est négligeable car le temps d'exécution est très petit. En revanche, dès que $N$ devient grand, on remarque que A1 prend un temps d'exécution plus grand que A2, qui prend un temps d'exécution plus grand que A3, qui prend un temps d'exécution plus grand que A4, ce qui vérifie la comparaison théorique entre les quatre algorithmes. Les cinq figures étant en échelle logarithmique sur les deux axes, la hiérarchie se lit comme des droites de pentes différentes : les points de A1 et A2 suivent une pente proche de 1 (croissance linéaire), tandis que ceux de A3 et A4 suivent une pente proche de 1/2 (croissance en racine carrée), ce qui est exactement le comportement prédit par le calcul de complexité.

## Conclusion

Pour conclure, la comparaison théorique et expérimentale des quatre algorithmes nous permet de tirer plusieurs enseignements. Sur le plan théorique, on remarque qu'A1 et A2 ont tous les deux une complexité en $O(N)$, mais avec des constantes différentes : $7N - 7$ instructions pour A1 et $4N - 8$ pour A2, ce qui divise par environ 1,75 le nombre d'instructions de A2 par rapport à A1 ; A3 et A4, quant à eux, ont une complexité en $O(\sqrt{N})$, avec $8\sqrt{N} - 8$ instructions pour A3 et $4\sqrt{N} + 1$ pour A4, ce qui divise par 2 le nombre d'itérations de A4 par rapport à A3. L'écart entre les algorithmes en $O(N)$ et ceux en $O(\sqrt{N})$ augmente avec $N$ : plus $N$ est grand, plus l'avantage d'A3 et d'A4 devient important. Sur le plan expérimental, les mesures confirment la hiérarchie théorique : on remarque que A1 prend un temps d'exécution plus grand que A2, qui prend un temps d'exécution plus grand que A3, qui prend un temps d'exécution plus grand que A4, ce qui vérifie la comparaison théorique entre les quatre algorithmes. Pour les petits nombres dans les cinq datasets (`Test-1` et `Test-2`), la différence de complexité est négligeable car le temps d'exécution est très petit, mais pour les grands nombres (`Random100`, `Random1000`, `Test-3`), l'écart devient spectaculaire : sur un nombre à 12 chiffres, A1 mettrait plusieurs heures alors qu'A3 et A4 terminent en quelques millisecondes. Le fait que le ratio $t/T(N)$ reste approximativement constant autour de 0,29 ns par unité d'instruction pour A1 montre que les prédictions théoriques sont compatibles avec les mesures expérimentales, et que la complexité observée est bien en $O(N)$. Sur le plan pratique, on peut retenir que pour un nombre de taille modérée n'importe quel algorithme convient, que pour des nombres jusqu'à $10^9$ on recommande A3 ou A4 (avec un avantage systématique pour A4, environ 2 fois plus rapide pour un coût d'implémentation négligeable), et que pour des nombres à 10 chiffres et plus seuls A3 et A4 sont praticables, A1 et A2 ne servant plus qu'à illustrer les notions de complexité.